

■ Full-Stack Movie Collection Ecosystem

Implementation Log & Architecture Review

Date: October 8, 2026 | Target: Production Environment

1. Monorepo Solution Overview

To ensure structural integrity and proper orchestration, all projects are organized under a single unified tracking monorepo workspace. This allows simultaneous delivery of backend serverless microservices, a frontend client interface, and continuous integration workflows.

Directory / File	Classification & Operational Objective
.github/workflows/web-app-ci.yml	Automated multi-project TypeScript compilation and verification gateway.
backend-worker/	Serverless Hono.js/TypeScript REST API routed completely on the Cloudflare global edge network.
frontend-ui/	React SPA interface leveraging Vite, TypeScript data contracts, and Axios network providers.
python-scraper/	Isolated automation layer pulling storefront listing data points using BeautifulSoup4.

2. Chronological Implementation Steps

Step 1: .NET Relational Entity Modeling

Created the core structural C# entity blueprint inside the isolated class library mapping properties to the specific Postgres database tracking table target.

```
[Table("Movie_Collection")]
public class Movie { ... }
```

Step 2: Package Dependency Synchronization

Resolved initial PMC package structure mismatched tool configurations by forcing global context assemblies to sync identical versions using localized installation switches.

```
Install-Package Microsoft.EntityFrameworkCore.Tools
Install-Package Microsoft.EntityFrameworkCore.Design
```

Step 3: Database Blueprint Generation

Compiled the data layer modeling assertions down into a local, isolated database script migration schema file.

```
Add-Migration InitialCollectionSetup -Project MovieCatalog.Data -StartupProject
MovieCatalog.API
```

Step 4: Local Network Connection Configuration

Overcame connection socket timeouts over direct Postgres networks by pointing strings to the secure transaction proxy engine route on port 6543 in Session Mode, appending Pooling=false; parameters to cleanly drop idle states.

Step 5: React Single Page Application Scaffolding

Scaffolds a clean frontend module workspace running over localized environments using modern Vite templates.

```
npm create vite@latest frontend-ui -- --template react-ts
npm install axios
```

Step 6: Serverless Architecture Transition (The Cloudflare Pivot)

To remain completely cloud-native and secure, the traditional .NET application framework was pivoted into a serverless TypeScript Cloudflare Worker managed by Hono.js. This eliminates cold starts and resource footprints entirely.

```
npm create hono@latest backend-worker
```

Step 7: Eliminating COUNTER Limit Subrequest Loops

Addressed the 'Too many subrequests' thread exception code (Cloudflare's maximum cap of 50 subrequests per invocation) by migrating from heavyweight stateful TCP socket drivers directly over to Supabase's stateless REST HTTP PostgREST API Gateway wrapper.

Step 8: Row-Level Security (RLS) Clearance and Mapping

Resolved the blank page rendering loops ('Total Movies cataloged: 0') by creating explicit SELECT and INSERT data security policies in Supabase, and mapping fields to case-insensitive contracts.

Step 9: GitHub Actions CI Quality Gates

Provisioned an automated verification workflow file (.github/workflows/web-app-ci.yml) compiling your serverless backend code on every push loop to enforce code stability.

```
npx tsc --noEmit
```

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.